Quiz

05/09/2026

NumPy

- library
- linear algebra, fourier transformation and Matrices. (Break in sine cosine frequency component)

why? → faster than list.

Numpy written in → Python, C or C++

→ Creating Array

```
import numpy as np
arr = np.array([[1,2], 4, 5])
print (arr)
print (type(arr))
```

* Array → 0 D → arr = np.array(42)

- 1 D → arr = np.array([1,2,3, ... 5])
- 2 D

* Check of dimension :→ ndim

eg. a = np.array (42)
print (a.ndim)

* Numpy Array Indexing

→ Access Array Element
└→ Just like list in Python.

```
arr = np.array [[1,2,3,4]]
print (arr[0])
```

output → 1

→ Access 2-D Array.

```
np.array([[1,2,3,4,5], [6,7,8,9,10]])
print(arr[0, 1])
```

[0] → row, [1] → element no.

Kaniwalikat

3d Arrays → [0, (1 2)]

0 → array
1 → which one
2 → index

* negative indexing → Just like Python.

slicing → Just like Python.
- it returns list.
- Just like Python.
  - * [:4] → not included.
  - * [0:5:2] → step
    - not included.

* 2D slicing:
  - arr [1, 1:4] → no. of list. → not executed
  - arr [0, 0:2, 2] → both

Data types in Python:

- strings → "HAI3CO"
- integer → -1, ...
- float → 1.2, +2.42
- boolean → True, or False
- complex → 1i + 2j, 1.5i + 2.5j

i - integer
b - boolean
u - unsigned integer
f - float
c → complex float
m - time delta
M → datetime
O - object
S - string
U - unicode string
V → fixed chunk of memory for other (void)

dtype → to data type of the array.

(5) → Method → for creation of Array.

---

(4) a = np.zeros((2,5))

array([[0., 0., 0., ...], [0 ...]]) 2x5 → array

* np.arrange(15) → (0, n-1)
* np.linspace(1,5,12) → Default No 12 → Number of equally spaced.

* np.empty((4,6)) → array dega (4x6) Random numbers honge

* np.empty_like(kpara) → array.
→ creates empty array.

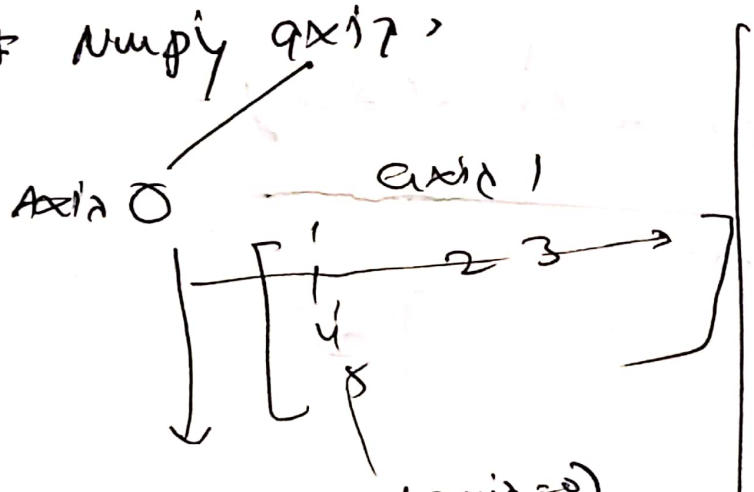
* np.identity(45) → (45x45)

* np.arange(99) → array → 0 —— 98

* arr.reshape(3,33)

* arr.ravel() → 1D Array

Numpy axis:



arr.sum(axis=0)
↓
var

* arr.T → transpose

* ar.ar.flat → to get rows ↓ transpose

* ar.ndim → rows dimension

* ar.nbytes → size.

arr.argmax() → kahape hai wo batata hai

dell
i7 11th gen
16 Ram
256 - storage → (SSD)

arr.argsort()
↓
sorted
arra(0,4,1,2,3)
↓
ascending

Mathematical

ar2, ar1 = [[- - - - -]
[- -]
[ ]],
[[ar]
[ ]
[ ]]

ar2 * ar1 =
ar.sqrt
ar.sum
ar.min
ar.max

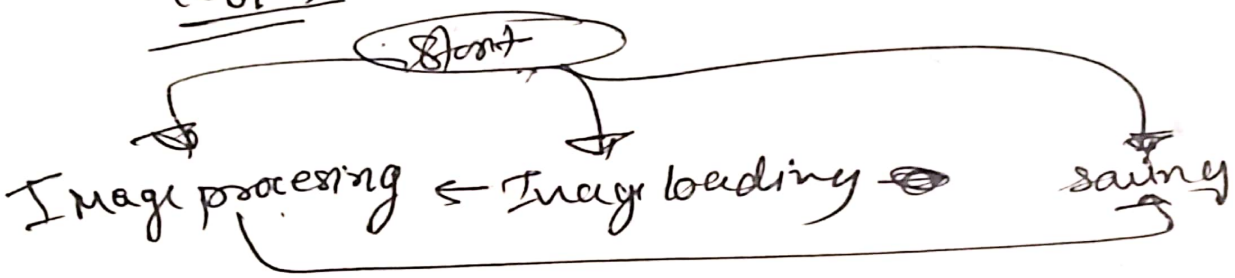
np.where (ar>5)
array([1,2], dtype)

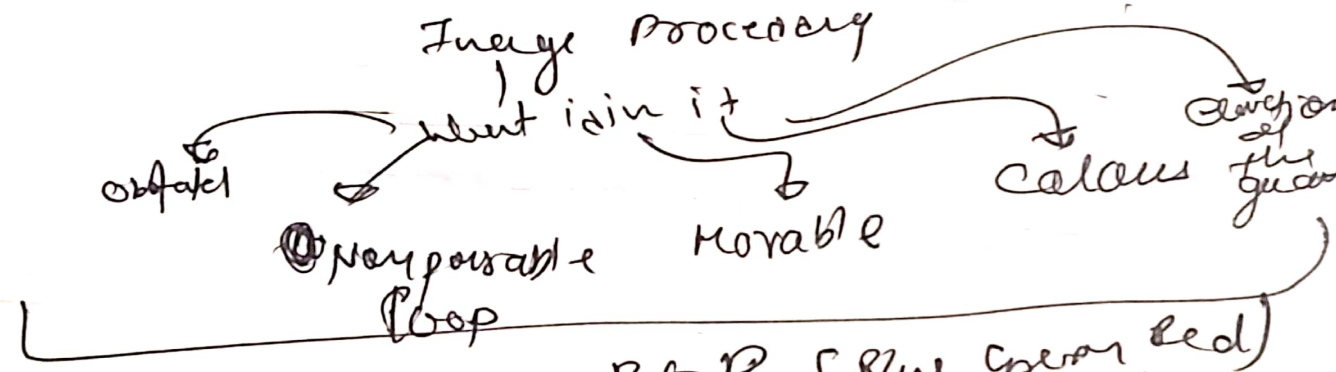
np.count_nonzero (ar)
→ count the non zeros

Open CV.

Through gesture for guide.

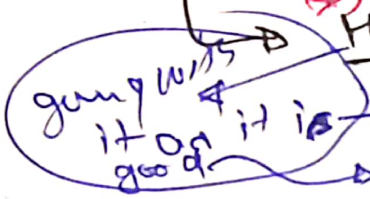
Tasks-1





#forget Colours → BGR (Blue Green Red) intensity

HSV (Hue, Saturation value)



Beacon I'll have Colours for direction

Images present in the Camera:

① wall → converted to Black which can't get recognised

V<60

Water Bodies → oval shaped this is obstacle then it can't go.

H = 101+140   S>180

elevation →

→ smaller area
→ mid area
→ large area

Area → 1<2<3

other colour in the image: & shapes

- red
- yellow
- white
- orange
- purple

shapes

- square
- star
- circle

because in top-view it gives more shades

so Area is given priority

Now deciding where it can go or where it can't go. and where it can't go → Black spot

→ get the map/way where we can go

● Shapes of the consider for calculations → Intensity

Scan:

- for start → orange
- and end → purple

purple vs blue ??

orange circle ??

distinguish with shape ?

(4) processing → shape identification → start stop

position → where to go?? & How

Calculation of scores.

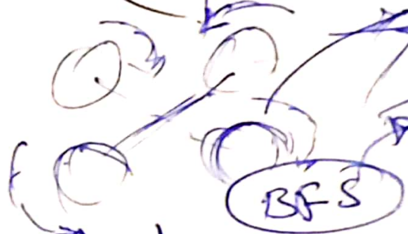
1) Priority score: Severity × Age score

2) [Path (carry) Calculate score] = $\left(\frac{\text{Displacement from start}}{\text{distance to well}}\right) \times P.S$

Path planning

→ A* Algorithm

→ was (Dijkstra)



Alpha-α

→ Breadth first search

diagonal Manhattan Euclidean better now

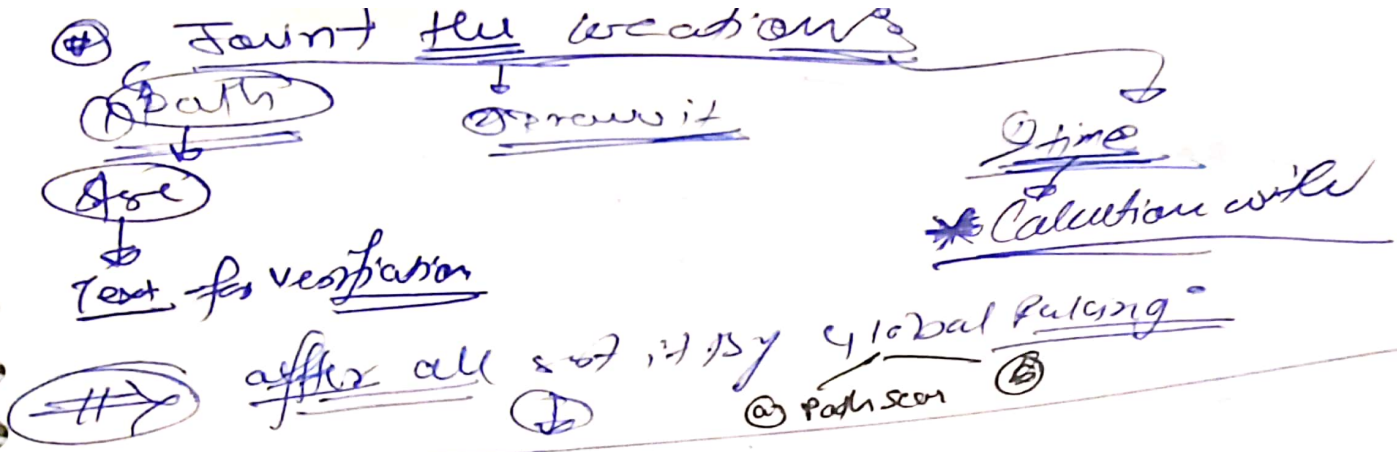
- 8 directions
- 4px or 5.65 px (unequal) → it cancels
- Uses a everytime that why because we need to check every step we

(1) going with greedy problem :-

→ as it checks only (n^2) order

(2) going with brute force :-

→ in this we have to check for 9! order



Task1 Output Report:

1. Traversable and Non-Traversable: Done
2. Casualty Information Done
3. Race Path: Done.
4. Path score: Done.
5. Path visualization: Done.
6. Time Calculation: Done.
7. Global Ruling: Done ⓐ ✓ ⓑ ✓

Task ② **Bonus**

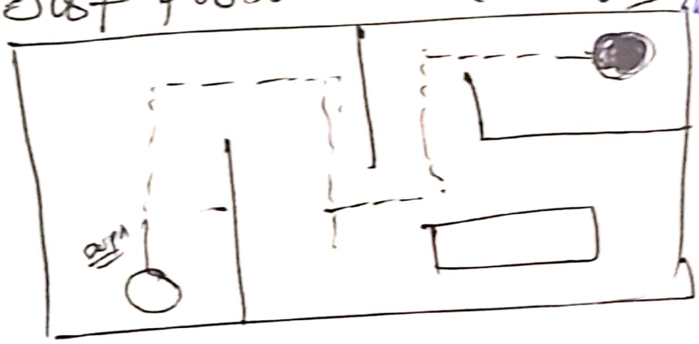
Restrictions: ① ek baar Hi dekh sakte Hai full Image.

② after that we can see only.

ek baar dekh ke Memory plotting karlo lek.

fir udh ni Jana,

Just possible all image:



Height x width/2

width/n x height/2

white

Task Completion Report:

- stopped the area of circle
- speed 20 px only
- every 20 px take a SS — By CV
- using open CV tool got the video

Wrong

Task: 2

The image was not Plain Image as it has Many pixel with green area # so its to fore [illegible]

Padega.

(1) Method (1) → Compressing the Image to Couple one color so that pixel get counter which are used.

(2) dimishing the Intensity and Enhancing the dark area

---

(1) input → find dimension → circle dimension & final detection correct.

3+1 = (4)
4500
4500 → 7000
[illegible] 4
[illegible]

(#) input Image → Kyuki data Hai Issue.

↓

* (Blur) tool → cv2.medianBlur (img, 5) → better.
* cv2.cvtColor (BGR → HSV) → to get accurate color fixen.
* get the area of circle.
* get all the data

0 — [illegible]
1 — [illegible]
2 → [illegible]

loops

Random unchosen the positions

Snap it

Ask whether reached? — Yes → completed

No

Check for surrounding and move

If Hit store the data and move & record the cordinate.

again follow the loop

Since distraction is a slight after image. future purpose Right hand is necessary

Testing Result:

Result completed:

Handle was : the image was dotted Pixelated
Rest was normal.

#UAS

Abhijeet

05/09/2020